07 · Directives, Class and Style Binding

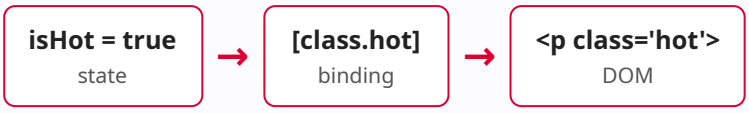
Beginner Angular learning notes

a. Definition

A **directive** is an instruction that changes how an element looks or behaves. There are three kinds: components (directives with a template), **structural** directives (change the DOM structure, like `@if`) and **attribute** directives (change appearance, like `NgClass`).

The easiest ways to style dynamically are **class binding** `[class.active]="cond"` and **style binding** `[style.color]="value"`. For many classes at once use `NgClass` / `NgStyle` with an object.

b. Diagram



A signal changes -> bindings recalculate -> DOM classes/styles update

c. Syntax

```
<p [class.active]="isActive">...</p>
<p [style.color]="color" [style.font-size.px]="size">...</p>
<p [ngClass]="{ big: isBig, small: !isBig }">...</p>
<p [ngStyle]="{ 'background-color': bg }">...</p>
```

d. Example Program

`Programs/07_Directives_and_Styling/dynamic-style.component.ts`

```
import { Component, computed, signal } from '@angular/core';
import { NgClass, NgStyle } from '@angular/common';

@Component({
  selector: 'app-root',
  imports: [NgClass, NgStyle],
  template: `
    <!-- class binding: add the "hot" class only when isHot() is true -->
    <p [class.hot]="isHot()">Tea is {{ isHot() ? 'hot' : 'cold' }}</p>

    <!-- NgClass with an object: several classes at once -->
    <p [ngClass]="{ big: size() > 16, small: size() <= 16 }">Size {{ size() }}px</p>

    <!-- style binding with a unit -->
    <p [style.font-size.px]="size()">This text resizes</p>

    <!-- NgStyle with an object -->
    <p [ngStyle]="{ color: textColor(), 'font-weight': 'bold' }">Colour: {{ textColor() }}</p>

    <button id="temp" (click)="isHot.set(!isHot())">Toggle temperature</button>
    <button id="size" (click)="size.update(s => s + 4)">Bigger text</button>
  `,
  styles: `
    .hot { color: crimson; }
```

```
    .big { text-decoration: underline; }
  `,
})
export class DynamicStyleComponent {
  isHot = signal(true);
  size = signal(14);
  // computed: derived value that updates automatically
  textColor = computed(() => (this.isHot() ? 'crimson' : 'royalblue'));
}
```

e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. Click **Toggle temperature**.
2. Click **Bigger text**.

```
Tea is cold
Size 18px
This text resizes
Colour: royalblue
Toggle temperatureBigger text
```

f. Real-World Example in Tamil (தமிழ் உதாரணம்)

பொங்கல் அன்று வீட்டு வாசலில் கோலம் போடுகிறோம். சாதாரண நாட்களில் சிறிய கோலம்; பண்டிகை நாளில் வண்ணக் கோலம். வாசல் அதே வாசல்தான், ஆனால் நாளுக்கு ஏற்ப அதன் தோற்றம் மாறுகிறது.

அதுபோல HTML element அதேதான்; `[class.hot]` அல்லது `[style.color]` மூலம் நிலைக்கு (state) ஏற்ப அதன் நிறம், அளவு, வடிவம் தானாக மாறும். நிலை மாறினால் தோற்றமும் உடனே மாறும் - நாம் கையால் DOM-ஐ மாற்ற வேண்டியதில்லை.

g. Common Mistakes

- Using `[class]="'a b'"` and accidentally removing other classes; prefer `[class.name]` for single classes.
- Forgetting units, e.g. `[style.font-size]="16"` instead of `[style.font-size.px]="16"`.
- Forgetting to import `NgClass` / `NgStyle` in a standalone component.
- Writing inline styles for everything instead of using CSS classes.

h. Interview Questions

Q1. What are structural vs attribute directives?
Structural directives add/remove elements (`@if`, `@for`); attribute directives change an element's look or behaviour (`NgClass`).

Q2. `[ngClass]` vs `[class.x]`?
`[class.x]` toggles one class with a boolean; `[ngClass]` handles many classes via object, array or string.

Q3. Is `*ngIf` still valid?
Yes but deprecated in favour of `@if`; new code should use the control flow blocks.

Q4. How do you bind a CSS unit?
Add the unit after the style name, e.g. `[style.width.px]` or `[style.height.%]`.

i. Practice Exercises

1. Make a dark/light theme toggle that changes the background and text colour of a card.
2. Highlight the row of a list item when the mouse clicks on it (use `[class.selected]`).
3. Create a progress bar whose width is bound to a number from an input slider.